

Python re Module

```
# =====
# PYTHON re MODULE - COMPLETE EXPLANATION IN ONE SCRIPT
# =====

import re

# =====
# 1. BASIC SEARCH
# =====

text = "I am learning Python for Data Engineering."
result = re.search("Python", text)

if result:
    print("Python found!")

print(result)

# =====
# 2. SEARCH FOR A WORD
# =====

text = "Ahmed is working as a Data Engineer."
result = re.search(r"Data Engineer", text)

if result:
    print("\nFound:", result.group())

# =====
# 3. search()
# =====

# search() finds the FIRST match anywhere in the string.

text = "Python SQL Python Spark"
result = re.search(r"Python", text)

print("\nsearch():")
print(result.group())

# =====
# 4. findall()
# =====

# findall() finds ALL matches.

text = "Python SQL Python Spark Python"
matches = re.findall(r"Python", text)

print("\nfindall():")
print(matches)

print("Number of matches:", len(matches))
```

```

# =====
# 5. finditer()
# =====

text = "Python SQL Python Spark"
matches = re.finditer(r"Python", text)

print("\nfinditer():")

for match in matches:
    print(
        "Value:", match.group(),
        "Start:", match.start(),
        "End:", match.end()
    )

# =====
# 6. MATCH FROM THE BEGINNING
# =====

text = "Python is powerful."
result = re.match(r"Python", text)

print("\nmatch():")

if result:
    print("Matched:", result.group())

# This will NOT match because Python
# is not at the beginning.

text = "I love Python."
result = re.match(r"Python", text)

print(result)

# =====
# 7. FULL STRING VALIDATION
# =====

text = "12345"
result = re.fullmatch(r"\d+", text)

print("\nfullmatch():")

if result:
    print("The entire string contains numbers.")

# =====
# 8. IMPORTANT REGEX SYMBOLS
# =====

"""
=====
COMMON REGEX PATTERNS
=====

.      → Any character
\d     → Digit

```

<code>\D</code>	→ Not a digit
<code>\w</code>	→ Letter, digit, underscore
<code>\W</code>	→ Not letter/digit/underscore
<code>\s</code>	→ Whitespace
<code>\S</code>	→ Not whitespace
<code>^</code>	→ Start of string
<code>\$</code>	→ End of string
<code>+</code>	→ One or more
<code>*</code>	→ Zero or more
<code>?</code>	→ Zero or one
<code>{n}</code>	→ Exactly n times
<code>{n,m}</code>	→ Between n and m times
<code>[]</code>	→ Character set
<code>()</code>	→ Group
<code> </code>	→ OR
<code>""</code>	

```
# =====
# 9. DIGITS
# =====
```

```
text = "Order 123 contains 45 items."
```

```
numbers = re.findall(r"\d+", text)
```

```
print("\nDigits:")
print(numbers)
```

```
# =====
# 10. LETTERS
# =====
```

```
text = "123 Ahmed 456 Mohamed"
```

```
letters = re.findall(r"[A-Za-z]+", text)
```

```
print("\nWords:")
print(letters)
```

```
# =====
# 11. SPECIFIC CHARACTER SET
# =====
```

```
text = "Ahmed123"
```

```
result = re.findall(r"[A-Za-z]", text)
```

```
print("\nLetters only:")
print(result)
```

```

# =====
# 12. LOWERCASE LETTERS
# =====

text = "Ahmed Python SQL"

result = re.findall(r"[a-z]+", text)

print("\nLowercase sequences:")
print(result)


# =====
# 13. UPPERCASE LETTERS
# =====

text = "Ahmed PYTHON SQL"

result = re.findall(r"[A-Z]+", text)

print("\nUppercase sequences:")
print(result)


# =====
# 14. EXACT NUMBER OF DIGITS
# =====

text = "123 4567 89 12345"

numbers = re.findall(r"\d{3}", text)

print("\nExactly 3 digits:")
print(numbers)


# =====
# 15. BETWEEN 3 AND 5 DIGITS
# =====

text = "12 123 1234 12345 123456"

numbers = re.findall(r"\d{3,5}", text)

print("\n3 to 5 digits:")
print(numbers)


# =====
# 16. ONE OR MORE DIGITS
# =====

text = "Order 123 has 45 products."

numbers = re.findall(r"\d+", text)

print("\nOne or more digits:")
print(numbers)


# =====
# 17. OPTIONAL CHARACTER
# =====

text = "color colour"

matches = re.findall(r"colou?r", text)

```

```

print("\nOptional character:")
print(matches)

# =====
# 18. START OF STRING ^
# =====

text = "Python is easy."

result = re.search(r"^Python", text)

print("\nStarts with Python:")

if result:
    print("Yes")

# =====
# 19. END OF STRING $
# =====

text = "I am learning Python"

result = re.search(r"Python$", text)

print("\nEnds with Python:")

if result:
    print("Yes")

# =====
# 20. OR OPERATOR |
# =====

text = "I use Python and SQL."

matches = re.findall(r"Python|SQL", text)

print("\nOR:")
print(matches)

# =====
# 21. GROUPS ()
# =====

text = "Ahmed:25"

result = re.search(r"(\w+):(\d+)", text)

print("\nGroups:")

if result:
    print("Full match:", result.group())
    print("Name:", result.group(1))
    print("Age:", result.group(2))

# =====
# 22. EXTRACT EMAIL
# =====

text = ""
Contact:

```

```

ahmed@gmail.com
mohamed@yahoo.com
omar@hotmail.com
"""

emails = re.findall(
    r"[\w.-]+@[\w.-]+\.\w+",
    text
)

print("\nEmails:")
print(emails)

# =====
# 23. VALIDATE EMAIL
# =====

email = "ahmed@gmail.com"

pattern = r"^\[\w.-]+\@[\w.-]+\.\w+$"

if re.fullmatch(pattern, email):

    print("\nValid email")

else:

    print("\nInvalid email")

# =====
# 24. EXTRACT PHONE NUMBERS
# =====

text = """
Ahmed: 01012345678
Mohamed: 01198765432
Omar: 01255555555
"""

phones = re.findall(
    r"\b01\d{9}\b",
    text
)

print("\nPhone numbers:")
print(phones)

# =====
# 25. REMOVE UNWANTED CHARACTERS
# =====

text = "Ahmed!!! Python@@@ SQL###"

clean_text = re.sub(
    r"[^A-Za-z0-9\s]",
    "",
    text
)

print("\nClean text:")
print(clean_text)

```

```

# =====
# 26. REPLACE MULTIPLE SPACES
# =====

text = "Ahmed    is    learning    Python."

clean_text = re.sub(
    r"\s+",
    " ",
    text
)

print("\nSpaces cleaned:")
print(clean_text)

# =====
# 27. REMOVE EXTRA SPACES
# =====

text = "    Ahmed is a Data Engineer    "

clean_text = text.strip()

print("\nStripped text:")
print(clean_text)

# =====
# 28. SPLIT USING REGEX
# =====

text = "Ahmed, Mohamed; Omar|Youssef"

names = re.split(
    r"[,;|]",
    text
)

print("\nSplit using regex:")
print(names)

# =====
# 29. CASE-INSENSITIVE SEARCH
# =====

text = "PYTHON Python python"

matches = re.findall(
    r"python",
    text,
    re.IGNORECASE
)

print("\nCase insensitive:")
print(matches)

# =====
# 30. COMPILE A REGEX
# =====

pattern = re.compile(r"\d+")

text = "Orders: 100, 200, 300"

numbers = pattern.findall(text)

```

```

print("\nCompiled pattern:")
print(numbers)

# =====
# 31. DATA ENGINEERING EXAMPLE
# =====

# Raw customer data

customers = [
    "Ahmed, ahmed@gmail.com, 01012345678",
    "Mohamed, mohamed@gmail.com, 01198765432",
    "Omar, invalid-email, 01255555555",
]

print("\n===== CUSTOMER ETL =====")

clean_customers = []

for record in customers:

    # EXTRACT

    parts = record.split(",")

    name = parts[0].strip()
    email = parts[1].strip()
    phone = parts[2].strip()

    # VALIDATE EMAIL

    email_pattern = r"^[\\w.-]+@[\\w.-]+\\.\\w+$"

    valid_email = bool(
        re.fullmatch(
            email_pattern,
            email
        )
    )

    # VALIDATE PHONE

    phone_pattern = r"^01\\d{9}$"

    valid_phone = bool(
        re.fullmatch(
            phone_pattern,
            phone
        )
    )

    # TRANSFORM

    name = re.sub(
        r"\\s+",
        " ",
        name
    )

    # LOAD VALID DATA

    if valid_email and valid_phone:

```



```

        clean_customers.append({
            "name": name,
            "email": email,
            "phone": phone
        })

print("\nClean customers:")

for customer in clean_customers:
    print(customer)

# =====
# 32. DATA CLEANING EXAMPLE
# =====

raw_data = [
    "  Ahmed   Mohamed  ",
    "Mohamed!!!",
    "  Omar@@@  ",
    "Youssef    Nady"
]

clean_data = []

for value in raw_data:
    # Remove special characters
    value = re.sub(
        r"[^A-Za-z\s]",
        "",
        value
    )

    # Replace multiple spaces
    value = re.sub(
        r"\s+",
        " ",
        value
    )

    # Remove leading/trailing spaces
    value = value.strip()

    clean_data.append(value)

print("\nCleaned data:")

for value in clean_data:
    print(value)

# =====
# 33. DATA EXTRACTION FROM LOGS
# =====

log = """
2026-09-06 INFO User Ahmed logged in
2026-09-06 ERROR Database connection failed
2026-09-07 INFO User Mohamed logged in
"""

```

```

# Extract dates
dates = re.findall(
    r"\d{4}-\d{2}-\d{2}",
    log
)

# Extract log levels
levels = re.findall(
    r"\b(INFO|ERROR)\b",
    log
)

print("\nLog dates:")
print(dates)

print("\nLog levels:")
print(levels)

# =====
# 34. EXTRACT IDs
# =====

text = """
customer_id=101
customer_id=205
customer_id=309
"""

ids = re.findall(
    r"customer_id=(\d+)",
    text
)

print("\nCustomer IDs:")
print(ids)

# =====
# 35. QUICK SUMMARY
# =====

print("""
=====
re MODULE – IMPORTANT FUNCTIONS
=====

re.search()
    Find first match anywhere.

re.match()
    Match only from beginning.

re.fullmatch()
    Entire string must match.

re.findall()
    Return all matches as a list.

re.finditer()
    Return match objects.

re.sub()
    Replace matches.

re.split()
    Split using a regex.

```

```
re.compile()  
    Create reusable regex pattern.
```

=====

MOST IMPORTANT PATTERNS

=====

```
\\d+  
    One or more digits
```

```
\\w+  
    One or more word characters
```

```
\\s+  
    One or more spaces
```

```
[A-Za-z]+  
    Letters
```

```
\\d{3}  
    Exactly 3 digits
```

```
\\d{3,5}  
    3 to 5 digits
```

```
^  
    Start
```

```
$  
    End
```

```
|  
    OR
```

```
(...)  
    Group
```

=====

DATA ENGINEERING USE CASES

=====

```
Raw Data  
  ↓  
Regex Extraction  
  ↓  
Validation  
  ↓  
Cleaning  
  ↓  
Transformation  
  ↓  
Clean Data  
""")
```